Project Overview: IMDB Movie Insights & Data Analytics by Vaibhav

- Content Trends

- **Goal:** Analyze production and genre evolution of movies & TV shows.
- **Key Questions:** Yearly release trends, genre shifts, correlation between movies & TV shows.
- **Metrics:** Titles per year, genre distribution, average ratings & votes.
- **Insights:** Growth/decline periods, genre dominance, audience rating patterns.

- Audience Targeting

- **Goal:** Examine family-friendly vs adult-oriented content.
- **Key Questions:** Volume and engagement trends, rating differences, genre patterns by certification.
- **Metrics:** Titles by category, avg ratings, votes, genre distribution.
- **Insights:** Shifts in focus, engagement gaps, genre-specific targeting trends.

```
import os
import re
import json
import numpy as np
import pandas as pd
from tqdm.auto import tqdm
from ipywidgets import interact

import plotly.io as pio
pio.renderers.default = 'iframe'


import warnings
warnings.filterwarnings("ignore")

df = pd.read_csv("action.csv")
df = df.dropna()

del df['gross_income']
del df['description']

df = df.sample(frac=1).reset_index(drop=True)

df.head()
```

	id	name	year	rating	certificate	duration	genre	votes	directors_id	directors_r
0	tt4429160	Red Shoes and the Seven Dwarfs	(2019)	6.4	PG	92 min	Animation, Action, Adventure	4,280	nm1406845,nm7464857,nm9045599	Sun Hong,Moo-H Jang,Young
1	tt1959563	The Hitman's Bodyguard	(2017)	6.9	R	118 min	Action, Comedy, Crime	227247	nm0400850	Patrick Hug
2	tt0030375	Little Orphan Annie	(1938)	7.0	Passed	60 min	Action, Crime, Drama	55	nm0391750	Ben Hol
3	tt10888708	The Sleepover	(2020)	5.6	TV-PG	100 min	Action, Adventure, Comedy	10,370	nm2586324	Trish
4	tt0419946	The Marine	(2006)	4.7	PG-13	92 min	Action, Comedy, Drama	34049	nm1669022	John Bo

Next steps: Generate code with df | New interactive sheet

2. Data Cleaning

2.1) Cleaning name, duration & votes

```
df['name'] = df['name'].str.strip()

df['duration'] = df['duration'].str.replace(' min','').str.replace(',','').astype(int)

df['vates'] = df['votes'].str.replace(',','').astype(float).fillna(0).astype(int)
```

2.2) Cleaning year Column

```
import re
from tqdm.auto import tqdm

start_year = []
end_year = []

for text in tqdm(df['year']):

    result = re.findall(r"\d{4}",text)

    if len(result) == 0:
        start_year.append(0)
        end_year.append(0)

    elif len(result) == 1:
        start_year.append(result[0])
        end_year.append(result[0])

    elif len(result) == 2:
        start_year.append(result[0])
        end_year.append(result[1])

    else:
        print(text)


df['start_year'] = start_year
df['start_year'] = df['start_year'].astype(int)

df['end_year'] = end_year
df['end_year'] = df['end_year'].astype(int)

del df['year']
del start_year
del end_year
```

```
100%  5517/5517 [00:00<00:00, 105077.58it/s]
```

2.3) Cleaning Director Names and IDs

```
df['directors_id'] = df['directors_id'].str.replace('/name/','').str.replace('/','').str.replace('Anonymous','nm0000000')
df['directors_name'] = df['directors_name'].str.replace('nm0000000', 'Anonymous')

director_ids_list = []
director_names_list = []


for director_ids, director_names in df[['directors_id','directors_name']].values:

    if len(director_ids.split(',')) != len(director_names.split(',')):
        director_ids_list.append(np.nan)
        director_names_list.append(np.nan)
    else:
        director_ids_list.append(director_ids)
        director_names_list.append(director_names)

df['directors_id'] = director_ids_list
df['directors_name'] = director_names_list

df = df.dropna()

dicrectors_dict = dict(zip(director_ids_list, director_names_list))

with open('directors.json','w') as f:
    json.dump(dicrectors_dict,f)

del director_ids_list
del director_names_list
del df['directors_name']
```

2.4) Mapping Star Names and IDs

```
df['stars_id'] = df['stars_id'].str.replace('Anonymous','nm0000000')
df['stars_name'] = df['stars_name'].str.replace('nm0000000', 'Anonymous')

stars_ids_list = []
stars_names_list = []


for stars_ids, stars_names in df[['stars_id','stars_name']].values:

    if len(stars_ids.split(',')) != len(stars_names.split(',')):
        stars_ids_list.append(np.nan)
        stars_names_list.append(np.nan)
    else:
        stars_ids_list.append(director_ids)
        stars_names_list.append(director_names)

df['stars_id'] = stars_ids_list
df['stars_name'] = stars_names_list

df = df.dropna()

stars_dict = dict(zip(stars_ids_list, stars_names_list))

with open('stars.json','w') as f:
    json.dump(dicrectors_dict,f)

del stars_names_list
del stars_ids_list
del df['stars_name']
```

```
df.head()
```

	id	name	rating	certificate	duration	genre	votes	directors_id	stars_id	vates
0	tt4429160	Red Shoes and the Seven Dwarfs	6.4	PG	92	Animation, Action, Adventure	4,280	nm1406845,nm7464857,nm9045599	nm0001239	4280
1	tt1959563	The Hitman's Bodyguard	6.9	R	118	Action, Comedy, Crime	227247	nm0400850	nm0001239	227247
2	tt0030375	Little Orphan Annie	7.0	Passed	60	Action, Crime, Drama	55	nm0391750	nm0001239	55
3	tt10888708	The Sleepover	5.6	TV-PG	100	Action, Adventure, Comedy	10,370	nm2586324	nm0001239	10370
4	tt0419946	The Marine	4.7	PG-13	92	Action, Comedy, Drama	34049	nm1669022	nm0001239	34049

Next steps: Generate code with df | New interactive sheet

3. Action Movies & Entertainment Trend Analysis

Business Context

The global entertainment industry has undergone significant transformation due to changes in audience preferences, technological advancements, and the rise of digital streaming platforms. Analyzing historical content data helps stakeholders understand long-term industry patterns and make informed decisions.

Problem Statement

Analyze how content production volume, genre distribution, and engagement have evolved over time to identify long-term trends in the movie and TV industry.

Objectives

- Understand growth patterns in content production
- Identify genre shifts across different decades

- Analyze changes in audience reception over time
- Highlight key industry milestones and transitions

Key Analytical Questions

- How has the number of released titles changed year over year?
- How has the number of released TV shows changed year over year?
- How has the number of released movies changed year over year?
- Is there any trend you can find between the number of movies and TV shows published over the years?¶
- Distribution of the number of movies among different genres over the years?

Key Metrics

- Number of titles released per year
- Average rating per year
- Total and average votes per year
- Genre frequency over time

Expected Insights

- Periods of rapid growth or decline
- Genre dominance by decade
- Shifts in audience rating behavior

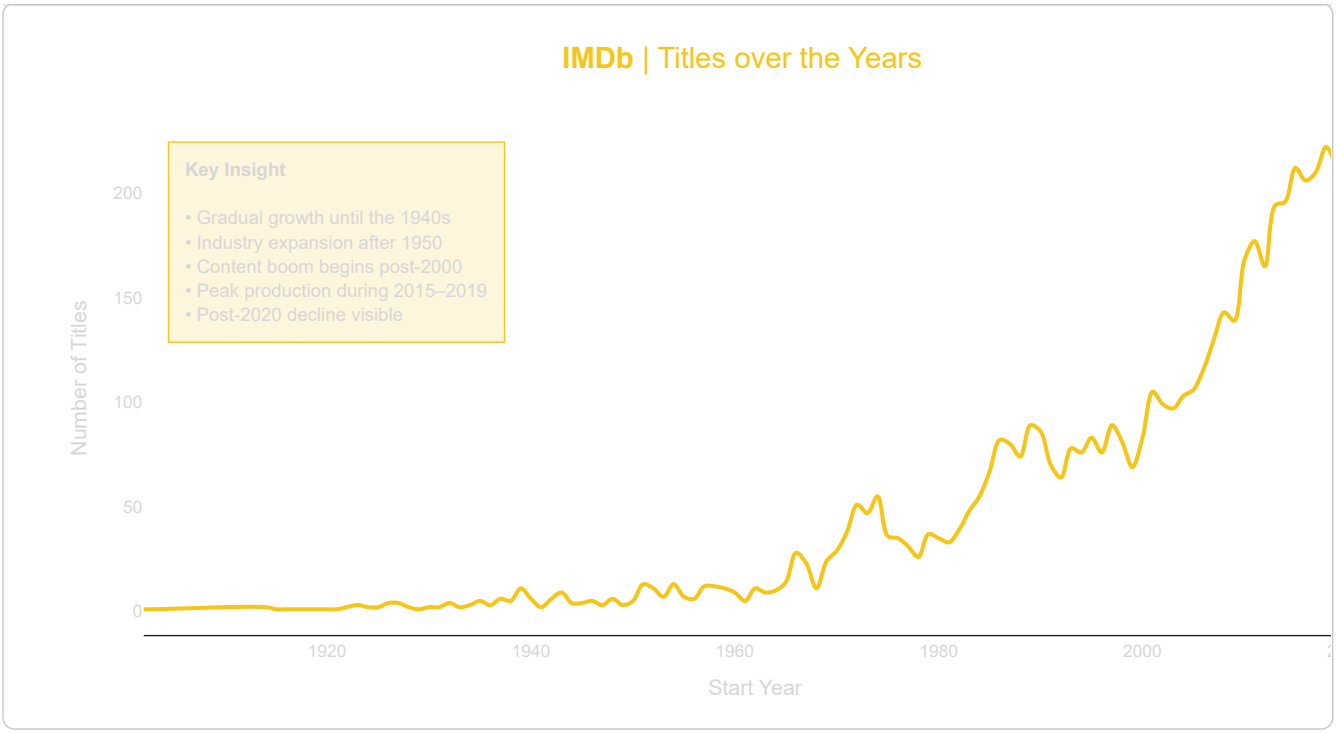
Deliverables

- Time-series visualizations
- Genre trend heatmaps
- Executive summary report
- Interactive dashboard (optional)

3.1) How has the number of released titles changed year over year?

```
import plotly.graph_objects as go
import plotly.io as pio
pio.renderers.default = 'colab'

# Filter data
df_ = df[(df['start_year'] > 1900) & (df['start_year'] < 2023)]

# Aggregate data
year_counts = df_['start_year'].value_counts().sort_index()

# IMDb dark theme colors
IMDB_YELLOW = '#F5C518'
IMDB_DARK = '#0F0F0F'
IMDB_GREY = '#1C1C1C'
TEXT_GREY = '#D6D6D6'
GRID_GREY = 'rgba(255,255,255,0.06)'
HIGHLIGHT_YELLOW = 'rgba(245,197,24,0.15)'

# Create figure
fig = go.Figure(
    go.Scatter(
        x=year_counts.index,
        y=year_counts.values,
        mode='lines',
        line=dict(
            color=IMDB_YELLOW,
            width=3,
            shape='spline'
        ),
        hovertemplate='<b>Year:</b> %{x}<br><b>Titles:</b> %{y}<extra></extra>'
    )
)

# Add conclusion annotation (top-left, integrated look)
fig.add_annotation(
    text=(
        "<b>Key Insight</b><br><br>"
        "• Gradual growth until the 1940s<br>"
        "• Industry expansion after 1950<br>"
        "• Content boom begins post-2000<br>"
        "• Peak production during 2015–2019<br>"
        "• Post-2020 decline visible"
```

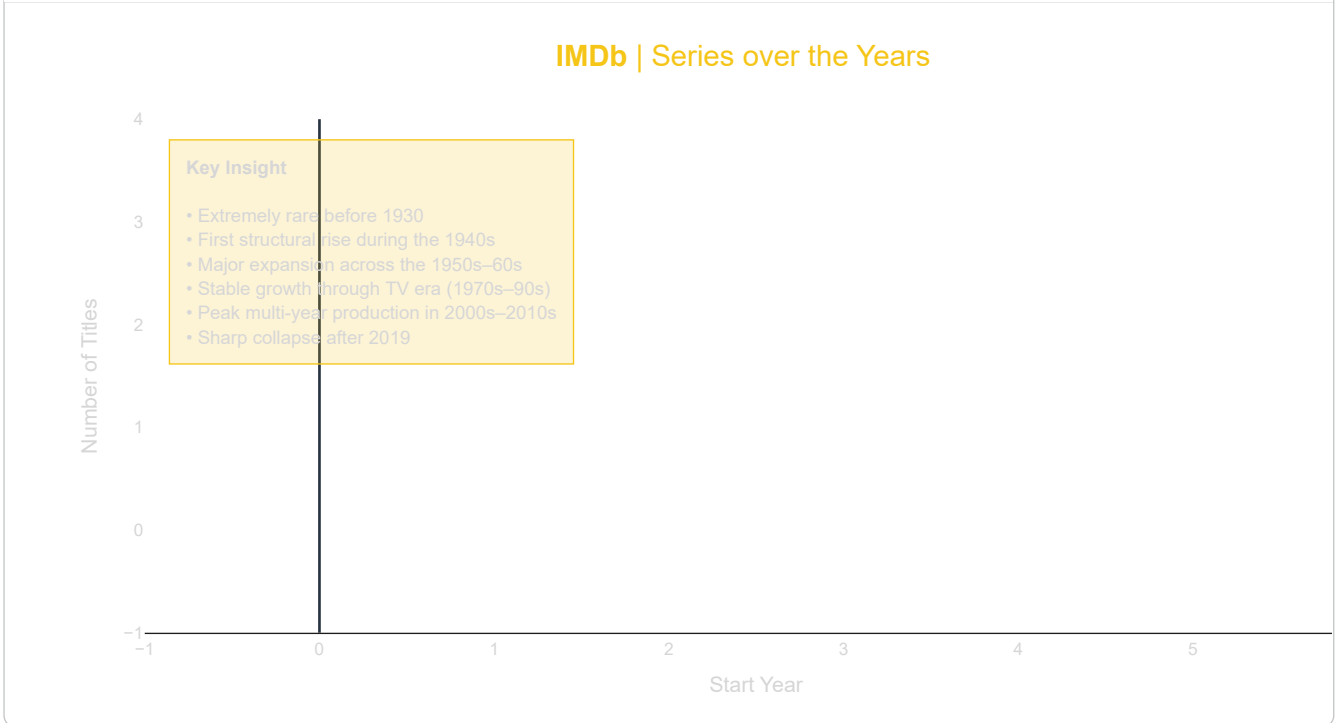
```
        ),
        xref='paper',
        yref='paper',
        x=0.02,
        y=0.96,
        align='left',
        showarrow=False,
        font=dict(
            family='Helvetica Neue, Arial, sans-serif',
            size=14,
            color=TEXT_GREY
        ),
        bgcolor=HIGHLIGHT_YELLOW,
        bordercolor=IMDB_YELLOW,
        borderwidth=1,
        borderpad=12
    )

    # Update layout
    fig.update_layout(
        title=dict(
            text='<b>IMDb</b> | Titles over the Years',
            x=0.5,
            font=dict(
                family='Helvetica Neue, Arial, sans-serif',
                size=22,
                color=IMDB_YELLOW
            )
        ),
        template='plotly_dark',
        font=dict(
            family='Helvetica Neue, Arial, sans-serif',
            size=13,
            color=TEXT_GREY
        ),
        xaxis=dict(
            title='Start Year',
            showgrid=False,
            linecolor=IMDB_GREY,
            tickcolor=TEXT_GREY
        ),
        yaxis=dict(
            title='Number of Titles',
            showgrid=True,
            gridcolor=GRID_GREY,
            zeroline=False
        ),
        plot_bgcolor=IMDB_DARK,
        paper_bgcolor=IMDB_DARK,
        margin=dict(l=60, r=40, t=80, b=60)
    )

    # Show plot
    fig.show()
```



3.2) How has the number of released TV shows changed year over year?

```
import plotly.graph_objects as go

# Filter data (titles spanning multiple years)
df_ = df[df['start_year'] != df['end_year']]
df_filtered = df_[(df_['start_year'] > 1850) & (df_['start_year'] < 2023)]
year_counts = df_filtered['start_year'].value_counts().sort_index()

# IMDb dark theme colors
IMDB_YELLOW = '#F5C518'
IMDB_DARK = '#0F0F0F'
IMDB_GREY = '#1C1C1C'
TEXT_GREY = '#D6D6D6'
GRID_GREY = 'rgba(255,255,255,0.06)'
HIGHLIGHT_YELLOW = 'rgba(245,197,24,0.18)'

# Create figure
fig = go.Figure(
    go.Scatter(
        x=year_counts.index,
        y=year_counts.values,
        mode='lines',
        line=dict(
            color=IMDB_YELLOW,
            width=3,
            shape='spline'
        ),
        hovertemplate='<b>Year:</b> %{x}<br><b>Titles:</b> %{y}<extra></extra>'
    )
)

# Updated conclusion annotation (data-driven)
fig.add_annotation(
    text=(
        "<b>Key Insight</b><br><br>"
        "• Extremely rare before 1930<br>"
        "• First structural rise during the 1940s<br>"
        "• Major expansion across the 1950s–60s<br>"
        "• Stable growth through TV era (1970s–90s)<br>"
        "• Peak multi-year production in 2000s–2010s<br>"
        "• Sharp collapse after 2019"
    ),
    xref='paper',
    yref='paper',
    x=0.02,
    y=0.96,
    align='left',
    showarrow=False,
    font=dict(
        family='Helvetica Neue, Arial, sans-serif',
        size=14,
```

```
            color=TEXT_GREY
        ),
        bgcolor=HIGHLIGHT_YELLOW,
        bordercolor=IMDB_YELLOW,
        borderwidth=1,
        borderpad=12
    )

    # Update layout
    fig.update_layout(
        title=dict(
            text='<b>IMDb</b> | Series over the Years',
            x=0.5,
            font=dict(
                family='Helvetica Neue, Arial, sans-serif',
                size=22,
                color=IMDB_YELLOW
            )
        ),
        template='plotly_dark',
        font=dict(
            family='Helvetica Neue, Arial, sans-serif',
            size=13,
            color=TEXT_GREY
        ),
        xaxis=dict(
            title='Start Year',
            showgrid=False,
            linecolor=IMDB_GREY,
            tickcolor=TEXT_GREY
        ),
        yaxis=dict(
            title='Number of Titles',
            showgrid=True,
            gridcolor=GRID_GREY,
            zeroline=False
        ),
        plot_bgcolor=IMDB_DARK,
        paper_bgcolor=IMDB_DARK,
        margin=dict(l=60, r=40, t=80, b=60)
    )

    # Show plot
    fig.show()
```

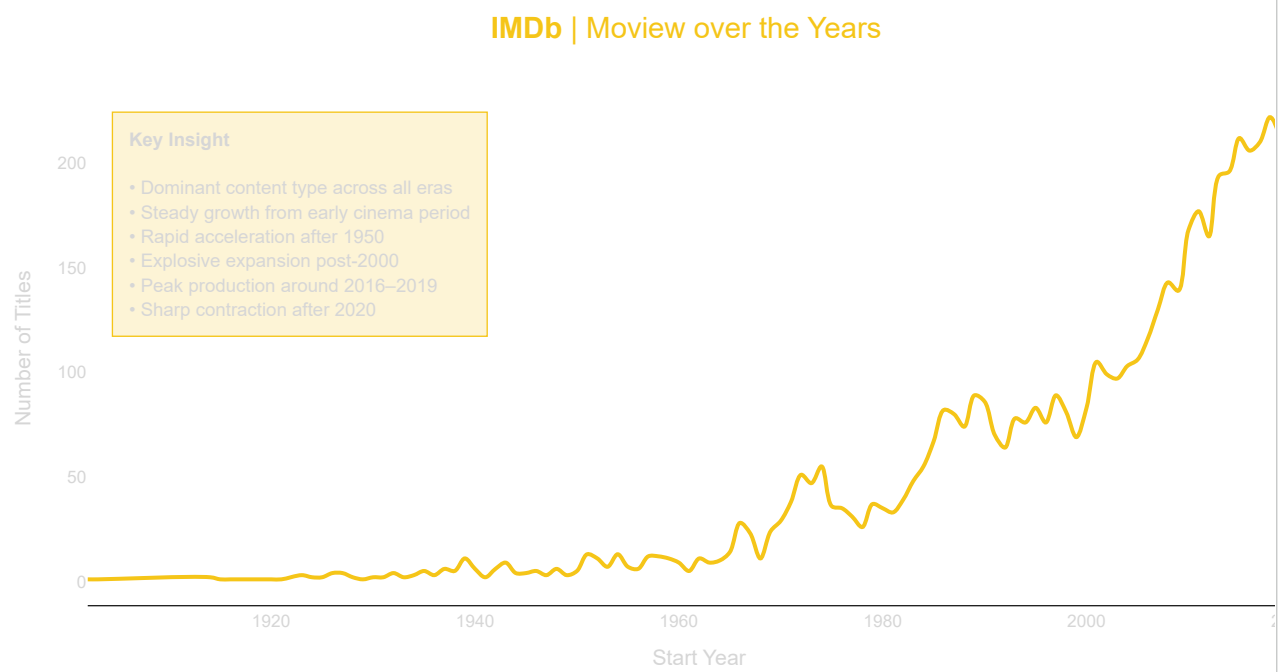


3.3) How has the number of released movies changed year over year?

```
    import plotly.graph_objects as go

    # Filter data (single-year titles)
    df_ = df[df['start_year'] == df['end_year']]
```

```
df_filtered = df_[(df_['start_year'] > 1850) & (df_['start_year'] < 2023)]
year_counts = df_filtered['start_year'].value_counts().sort_index()

# IMDb dark theme colors
IMDB_YELLOW = '#F5C518'
IMDB_DARK = '#0F0F0F'
IMDB_GREY = '#1C1C1C'
TEXT_GREY = '#D6D6D6'
GRID_GREY = 'rgba(255,255,255,0.06)'
HIGHLIGHT_YELLOW = 'rgba(245,197,24,0.18)'

# Create figure
fig = go.Figure(
    go.Scatter(
        x=year_counts.index,
        y=year_counts.values,
        mode='lines',
        line=dict(
            color=IMDB_YELLOW,
            width=3,
            shape='spline'
        ),
        hovertemplate='<b>Year:</b> %{x}<br><b>Titles:</b> %{y}<extra></extra>'
    )
)

# Conclusion annotation (top-left, integrated with plot)
fig.add_annotation(
    text=(
        "<b>Key Insight</b><br><br>"
        "• Dominant content type across all eras<br>"
        "• Steady growth from early cinema period<br>"
        "• Rapid acceleration after 1950<br>"
        "• Explosive expansion post-2000<br>"
        "• Peak production around 2016–2019<br>"
        "• Sharp contraction after 2020"
    ),
    xref='paper',
    yref='paper',
    x=0.02,
    y=0.96,
    align='left',
    showarrow=False,
    font=dict(
        family='Helvetica Neue, Arial, sans-serif',
        size=14,
        color=TEXT_GREY
    ),
    bgcolor=HIGHLIGHT_YELLOW,
    bordercolor=IMDB_YELLOW,
    borderwidth=1,
    borderpad=12
)

# Update layout
fig.update_layout(
    title=dict(
        text='<b>IMDb</b> | Moview over the Years',
        x=0.5,
        font=dict(
            family='Helvetica Neue, Arial, sans-serif',
            size=22,
            color=IMDB_YELLOW
        )
    ),
    template='plotly_dark',
    font=dict(
        family='Helvetica Neue, Arial, sans-serif',
        size=13,
        color=TEXT_GREY
    ),
    xaxis=dict(
        title='Start Year',
        showgrid=False,
        linecolor=IMDB_GREY,
        tickcolor=TEXT_GREY
    ),
    yaxis=dict(
        title='Number of Titles',
        showgrid=True,
        gridcolor=GRID_GREY,
        zeroline=False
```

```
    ),
    plot_bgcolor=IMDB_DARK,
    paper_bgcolor=IMDB_DARK,
    margin=dict(l=60, r=40, t=80, b=60)
)

# Show plot
fig.show()
```

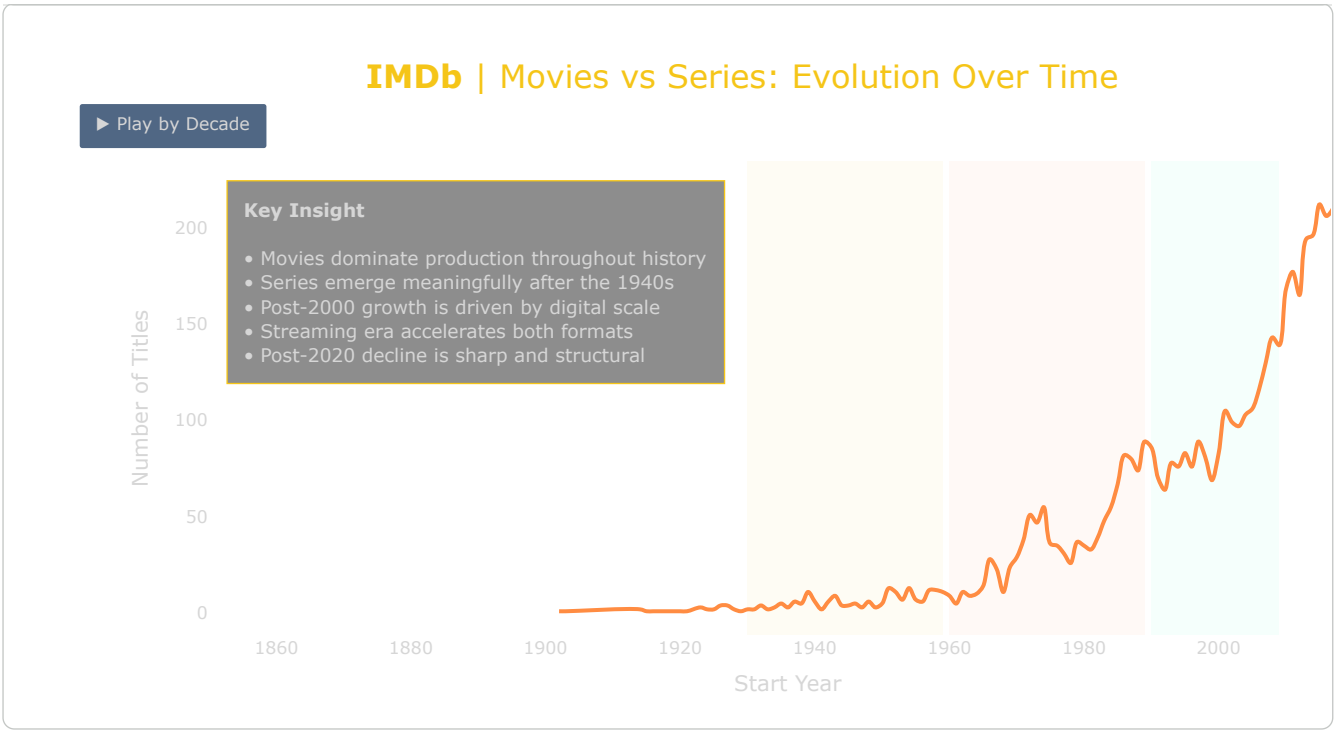


3.4) Is there any trend you can find between the number of movies and TV shows published over the years?¶

```
import plotly.graph_objects as go
import numpy as np

# ------------------------------
# Data preparation
# ------------------------------
df_diff = df[(df['start_year'] != df['end_year']) & (df['start_year'] > 1850) & (df['start_year'] < 2023)]
df_same = df[(df['start_year'] == df['end_year']) & (df['start_year'] > 1850) & (df['start_year'] < 2023)]

year_counts_diff = df_diff['start_year'].value_counts().sort_index()
year_counts_same = df_same['start_year'].value_counts().sort_index()

years = np.arange(1850, 2023)

# ------------------------------
# IMDb cinematic palette
# ------------------------------
IMDB_YELLOW = '#F5C518'     # Series
IMDB_AMBER = '#FF8C42'      # Movies
IMDB_DARK = '#0F0F0F'
TEXT_GREY = '#D6D6D6'
GRID_GREY = 'rgba(255,255,255,0.06)'

# Era shading colors
ERAS = [
    (1850, 1929, 'rgba(255,255,255,0.04)', 'Early Cinema'),
    (1930, 1959, 'rgba(245,197,24,0.05)', 'Studio Era'),
    (1960, 1989, 'rgba(255,140,66,0.05)', 'Television Era'),
    (1990, 2009, 'rgba(0,255,200,0.04)', 'Digital Expansion'),
    (2010, 2022, 'rgba(255,255,255,0.07)', 'Streaming Era')
]

# ------------------------------
# Base figure
# ------------------------------
fig = go.Figure()

fig.add_trace(
    go.Scatter(
        x=year_counts_diff.index,
        y=year_counts_diff.values,
```

```
        mode='lines',
        name='Series',
        line=dict(color=IMDB_YELLOW, width=3, shape='spline')
    )
)

fig.add_trace(
    go.Scatter(
        x=year_counts_same.index,
        y=year_counts_same.values,
        mode='lines',
        name='Movies',
        line=dict(color=IMDB_AMBER, width=3, shape='spline')
    )
)

# ----------------------------
# Gradient era shading
# ----------------------------
shapes = []
for start, end, color, _ in ERAS:
    shapes.append(
        dict(
            type='rect',
            xref='x',
            yref='paper',
            x0=start,
            x1=end,
            y0=0,
            y1=1,
            fillcolor=color,
            line=dict(width=0),
            layer='below'
        )
    )

# ----------------------------
# Animated frames (decades)
# ----------------------------
frames = []
for decade in range(1850, 2030, 10):
    frames.append(
        go.Frame(
            name=str(decade),
            data=[
                go.Scatter(
                    x=year_counts_diff.index[year_counts_diff.index <= decade],
                    y=year_counts_diff.values[year_counts_diff.index <= decade]
                ),
                go.Scatter(
                    x=year_counts_same.index[year_counts_same.index <= decade],
                    y=year_counts_same.values[year_counts_same.index <= decade]
                )
            ]
        )
    )

fig.frames = frames

# ----------------------------
# Conclusion annotation
# ----------------------------
fig.add_annotation(
    text=(
        "<b>Key Insight</b><br><br>"
        "• Movies dominate production throughout history<br>"
        "• Series emerge meaningfully after the 1940s<br>"
        "• Post-2000 growth is driven by digital scale<br>"
        "• Streaming era accelerates both formats<br>"
        "• Post-2020 decline is sharp and structural"
    ),
    xref='paper',
    yref='paper',
    x=0.015,
    y=0.96,
    align='left',
    showarrow=False,
    font=dict(size=14, color=TEXT_GREY),
    bgcolor='rgba(0,0,0,0.45)',
    bordercolor=IMDB_YELLOW,
    borderwidth=1,
    borderpad=12
```

```
)

# ----------------------------
# Layout (portfolio-ready)
# ----------------------------
fig.update_layout(
    title=dict(
        text='<b>IMDb</b> | Movies vs Series: Evolution Over Time',
        x=0.5,
        font=dict(size=24, color=IMDB_YELLOW)
    ),
    template='plotly_dark',
    plot_bgcolor=IMDB_DARK,
    paper_bgcolor=IMDB_DARK,
    shapes=shapes,
    font=dict(color=TEXT_GREY, size=13),
    xaxis=dict(
        title='Start Year',
        showgrid=False,
        range=[1850, 2022]
    ),
    yaxis=dict(
        title='Number of Titles',
        showgrid=True,
        gridcolor=GRID_GREY,
        zeroline=False
    ),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=1.02,
        xanchor='right',
        x=1
    ),
    updatemenus=[
        dict(
            type='buttons',
            showactive=False,
            x=0.05,
            y=1.12,
            buttons=[
                dict(
                    label='▶ Play by Decade',
                    method='animate',
                    args=[
                        None,
                        dict(
                            frame=dict(duration=400, redraw=True),
                            transition=dict(duration=300),
                            fromcurrent=True
                        )
                    ]
                )
            ]
        )
    ],
    margin=dict(l=60, r=40, t=110, b=60)
)

fig.show()
```

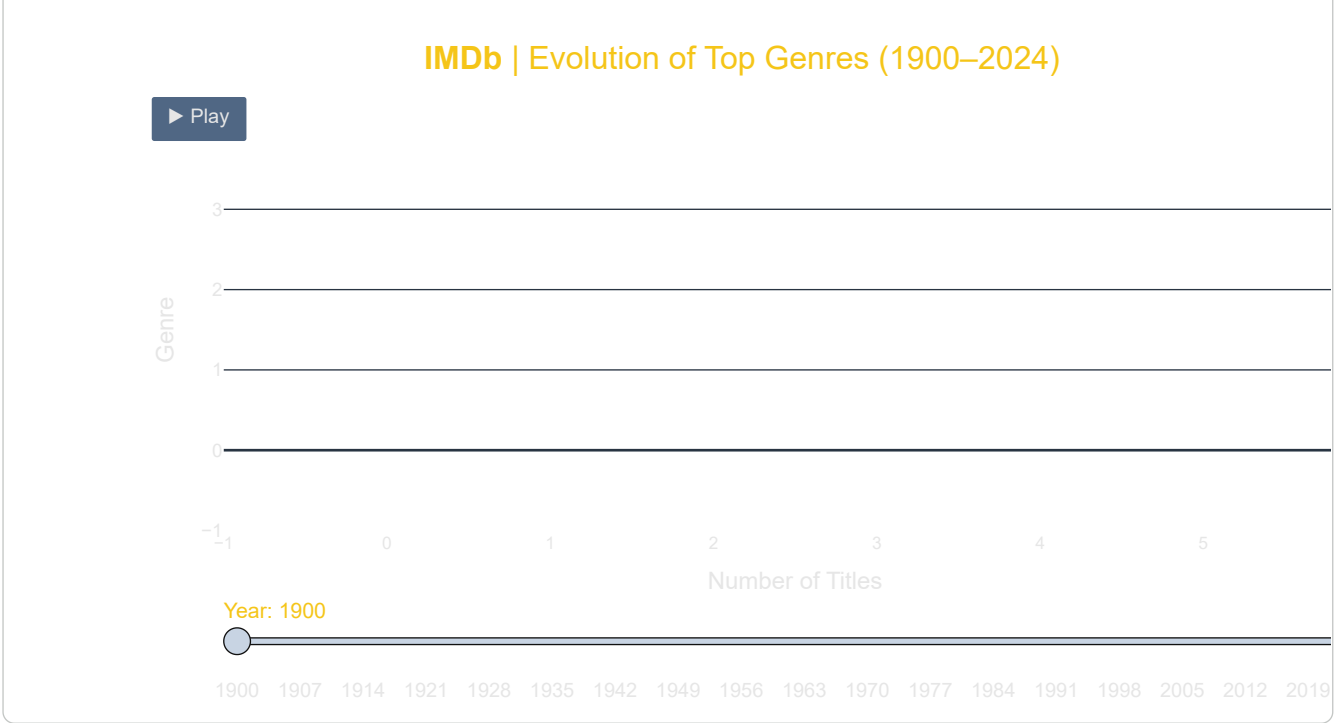


3.5) Distribution of the number of movies among different genres over the years?

```
import plotly.graph_objects as go

# ----------------------------
# Prepare years for animation
# ----------------------------
years = list(range(1900, 2025))

frames = []
for year in years:
    genres = (
        df[df['start_year'] == year]['genre']
        .dropna()
        .str.split(',')
        .explode()
        .str.strip()
        .value_counts()
        .head(20)
    )

    frames.append(
        go.Frame(
            data=[
                go.Bar(
                    x=genres.values[::-1],
                    y=genres.index[::-1],
                    orientation='h',
                    marker=dict(
                        color=genres.values[::-1],
                        colorscale=[[0, '#F5C518'], [1, '#FF8C42']],
                        line=dict(width=0)
                    ),
                    hovertemplate='<b>%{y}</b><br>Titles: %{x}<extra></extra>'
                )
            ],
            name=str(year)
        )
    )

# ----------------------------
# Initial frame
# ----------------------------
initial_year = years[0]
initial_genres = (
    df[df['start_year'] == initial_year]['genre']
    .dropna()
    .str.split(',')
    .explode()
    .str.strip()
    .value_counts()
```

```
    .head(20)
)

# ----------------------------
# Create figure
# ----------------------------
fig = go.Figure(
    data=[
        go.Bar(
            x=initial_genres.values[::-1],
            y=initial_genres.index[::-1],
            orientation='h',
            marker=dict(
                color=initial_genres.values[::-1],
                colorscale=[[0, '#F5C518'], [1, '#FF8C42']]
            ),
            hovertemplate='<b>%{y}</b><br>Titles: %{x}<extra></extra>'
        )
    ],
    frames=frames
)

# ----------------------------
# Layout – IMDb editorial dark
# ----------------------------
fig.update_layout(
    title=dict(
        text='<b>IMDb</b> | Evolution of Top Genres (1900–2024)',
        x=0.5,
        font=dict(size=24, color='#F5C518')
    ),
    template='plotly_dark',
    plot_bgcolor='#0F0F0F',
    paper_bgcolor='#0F0F0F',
    font=dict(
        family='Helvetica Neue, Arial, sans-serif',
        size=15,
        color='#E6E6E6'
    ),
    xaxis=dict(
        title='Number of Titles',
        showgrid=True,
        gridcolor='rgba(255,255,255,0.08)',
        tickfont=dict(size=13)
    ),
    yaxis=dict(
        title='Genre',
        tickfont=dict(size=14),
        automargin=True
    ),
    margin=dict(l=120, r=40, t=90, b=60),
    updatemenus=[
        dict(
            type='buttons',
            showactive=False,
            x=0.02,
            y=1.08,
            buttons=[
                dict(
                    label='▶ Play',
                    method='animate',
                    args=[
                        None,
                        dict(
                            frame=dict(duration=500, redraw=True),
                            transition=dict(duration=300),
                            fromcurrent=True
                        )
                    ]
                )
            ]
        )
    ]
)

# ----------------------------
# Slider
# ----------------------------
fig.update_layout(
    sliders=[
        dict(
            active=0,
```

```
            currentvalue=dict(
                prefix='Year: ',
                font=dict(size=16, color='#F5C518')
            ),
            pad=dict(t=45),
            steps=[
                dict(
                    method='animate',
                    label=str(year),
                    args=[
                        [str(year)],
                        dict(
                            mode='immediate',
                            frame=dict(duration=500, redraw=True),
                            transition=dict(duration=300)
                        )
                    ]
                )
                for year in years
            ]
        )
    ]
)

fig.show()
```



4. Evolution of Family vs Adult Content¶

Business Context

Audience targeting is a critical strategic decision for content creators and streaming platforms. Content certification helps classify titles as family-friendly or adult-oriented, influencing reach, engagement, and platform positioning.

Problem Statement

Analyze how family-friendly and adult-oriented content has evolved over time in terms of production volume, ratings, and audience engagement.

Objectives

- Compare growth trends of family vs adult content
- Evaluate differences in audience reception

Key Analytical Questions

- How does the volume of kids vs family vs adult content change over time?
- How does the engagement of kids vs family vs adult content change over time?
- Do we have any pattern in the distribution of certificate movies within each certificate_category?

- Are specific certificate movies driving engagement within each certificate_category?

Key Metrics

- Number of titles by certification group
- Average rating by certification category
- Votes per title by category
- Genre distribution within each category

Analytical Approach

- Group certifications into family-friendly and adult-oriented
- Analyze trends across time periods
- Compare performance metrics between groups
- Drill down into genre-level differences

Expected Insights

- Shifts toward adult or family-focused content
- Rating and engagement gaps between audience types
- Genre-specific audience targeting patterns

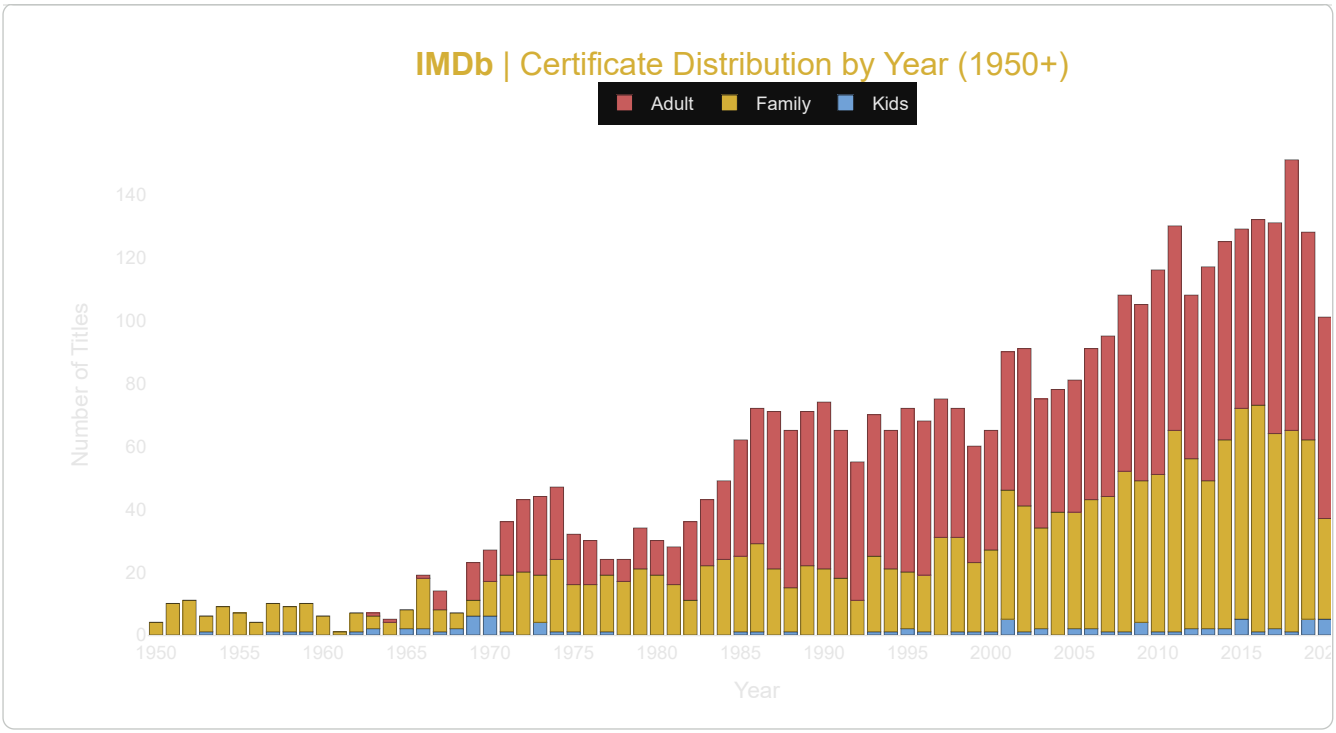
Deliverables

- Comparative trend charts
- Certification-based performance dashboards
- Strategic insights summary

4.1) How does the volume of kids vs family vs adult content change over time?

```
import plotly.graph_objects as go

# ------------------------------------------------
# Certificate grouping
# ------------------------------------------------
kids      = ['TV-Y', 'TV-Y7', 'TV-Y7-FV', 'TV-G', 'G','E', 'EC', 'CE', 'K-A', '6+']
family    = ['PG', 'TV-PG', 'PG-12', 'PG-13', 'GP', 'GA','Approved', 'Passed', 'Open', 'TV-14', '12','TV-13', 'E10+', 'T',
adult     = ['R', 'R-12', 'R-15', 'R-18', 'NC-17', 'X','AO', 'MA-13', 'MA-17', 'TV-MA', 'M','18', 'Banned', '(Banned)']
not_rated = ['Not Rated','Not Certified', 'Unrated']

cert_map = {c: 'Kids' for c in kids}
cert_map.update({c: 'Family' for c in family})
cert_map.update({c: 'Adult' for c in adult})
cert_map.update({c: 'Not Rated' for c in not_rated})

df['certificate_category'] = df['certificate'].map(cert_map)

# Filter usable data
import plotly.graph_objects as go
import pandas as pd

# ------------------------------------------------
# Filter data from 1950 onwards
# ------------------------------------------------
df_ = df[
    (df['certificate_category'].notna()) &
    (df['certificate_category'] != 'Not Rated') &
    (df['start_year'] >= 1950) &
    (df['start_year'] <= 2023)
]

years = sorted(df_['start_year'].unique())
categories = ['Kids', 'Family', 'Adult']

# ------------------------------------------------
# Cinematic muted colors
# ------------------------------------------------
category_colors = {
    'Kids':   '#70A1D7',   # muted blue
    'Family': '#D4AF37',   # warm gold
    'Adult':  '#C75C5C'    # soft red
}

# ------------------------------------------------
# Aggregate data for stacked bar chart
# ------------------------------------------------
```

```
agg_data = df_.groupby(['start_year', 'certificate_category']).size().unstack(fill_value=0)
agg_data = agg_data[categories]  # ensure consistent order

# ------------------------------------------------
# Create figure
# ------------------------------------------------
fig = go.Figure()

for cat in categories:
    fig.add_trace(
        go.Bar(
            x=agg_data.index,
            y=agg_data[cat],
            name=cat,
            marker_color=category_colors[cat],
            hovertemplate='<b>%{x}</b><br>' + cat + ': %{y}<extra></extra>'
        )
    )

# ------------------------------------------------
# Layout – IMDb cinematic style
# ------------------------------------------------
fig.update_layout(
    barmode='stack',
    title=dict(
        text='<b>IMDb</b> | Certificate Distribution by Year (1950+)',
        x=0.5,
        font=dict(size=24, color='#D4AF37')
    ),
    template='plotly_dark',
    plot_bgcolor='#0F0F0F',
    paper_bgcolor='#0F0F0F',
    font=dict(family='Helvetica Neue, Arial, sans-serif', size=14, color='#E6E6E6'),
    xaxis=dict(
        title='Year',
        showgrid=False,
        tickmode='linear',
        tick0=1950,
        dtick=5,
        linecolor='#E6E6E6'
    ),
    yaxis=dict(
        title='Number of Titles',
        showgrid=True,
        gridcolor='rgba(255,255,255,0.08)',
        zeroline=False
    ),
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=1.02,
        xanchor='center',
        x=0.5,
        font=dict(size=14)
    ),
    margin=dict(l=60, r=40, t=90, b=60)
)

fig.show()
```



4.2) How does the engagement of kids vs family vs adult content change over time?

```
import plotly.graph_objects as go
import pandas as pd

# ------------------------------------------------
# Filter data from 1950 onwards and remove Not Rated
# ------------------------------------------------
df_ = df[
    (df['certificate_category'].notna()) &
    (df['certificate_category'] != 'Not Rated') &
    (df['start_year'] >= 1950) &
    (df['start_year'] <= 2023)
]

years = sorted(df_['start_year'].unique())
categories = ['Kids', 'Family', 'Adult']

# ------------------------------------------------
# Cinematic muted colors
# ------------------------------------------------
category_colors = {
    'Kids':   '#70A1D7',   # muted blue
    'Family': '#D4AF37',   # warm gold
    'Adult':  '#C75C5C'    # soft red
}

# Convert votes to numeric
df_['votes'] = pd.to_numeric(df_['votes'], errors='coerce').fillna(0)

# ------------------------------------------------
# Aggregate votes for stacked bar chart
# ------------------------------------------------
agg_votes = df_.groupby(['start_year', 'certificate_category'])['votes'].sum().unstack(fill_value=0)
agg_votes = agg_votes[categories]  # ensure consistent order

# ------------------------------------------------
# Create figure
# ------------------------------------------------
fig = go.Figure()

for cat in categories:
    fig.add_trace(
        go.Bar(
            x=agg_votes.index,
            y=agg_votes[cat],
            name=cat,
            marker_color=category_colors[cat],
            hovertemplate='<b>%{x}</b><br>' + cat + ': %{y:,} votes<extra></extra>'
        )
    )
```

```
# ------------------------------------------------
# Layout – IMDb cinematic style
# ------------------------------------------------
fig.update_layout(
    barmode='stack',
    title=dict(
        text='<b>IMDb</b> | Total Votes by Certificate Category (1950+)',
        x=0.5,
        font=dict(size=24, color='#D4AF37')
    ),
    template='plotly_dark',
    plot_bgcolor='#0F0F0F',
    paper_bgcolor='#0F0F0F',
    font=dict(family='Helvetica Neue, Arial, sans-serif', size=14, color='#E6E6E6'),
    xaxis=dict(
        title='Year',
        showgrid=False,
```